Reverse Engineering Verilog to C to identify Trojan

A B. Tech Project Report Submitted
in Partial Fulfillment of the Requirements
for the Degree of

Bachelor of Technology

by

Achhada Hiren Rajkumar
(2101CS03)

under the guidance of

Dr. Somanath Tripathy



to the

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY PATNA
PATNA - 800013, BIHAR

CERTIFICATE

*This is to certify that the work contained in this thesis entitled "**Reverse Engineering Verilog to C to identify Trojan**" is a bonafide work of **Achhada Hiren Rajkumar** (**Roll No. 2101CS03**), carried out in the Department of Computer Science and Engineering, Indian Institute of Technology Patna under my supervision and that it has not been submitted elsewhere for a degree.*

Supervisor: **Dr. Somanath Tripathy**

Professor,

May, 2025

Department of Computer Science & Engineering,

Patna.

Indian Institute of Technology Patna, Bihar.

Acknowledgements

I extend our heartfelt gratitude to Prof. Dr. Somanth Tripathy, whose unwavering guidance and invaluable advice have been instrumental in bringing this project to fruition. His mentorship has been a beacon of support throughout its development.His constructive contributions have greatly enriched our work.

My deepest gratitude goes to our families and friends for their steadfast encouragement and unwavering belief in our abilities. Their love, motivation, and understanding have been constant sources of strength, propelling us forward on our academic journey.

Lastly, I acknowledge the divine guidance of God, whose presence and grace have sus tained us through every challenge we have encountered. I am profoundly grateful for His blessings and guidance, which have illuminated our path each step of the way.

Contents

List of Figures **vii**

List of Tables **ix**

1 Introduction **1**

1.1 Background 1

1.2 Contribution 3

2 Review of Prior Works **5**

2.1 Review of Prior Work 5

3 Dataset Details **7**

3.1 Dataset Construction and Preprocessing 7

3.1.1 C Code Preprocessing 8

3.1.2 Verilog Code Preprocessing 9

3.1.3 Program Consolidation and Data Preprocessing 9

3.1.4 Model Training and Generation 9

4 ML Models **11**

4.1 Main Architecture and Models Used 11

4.1.1 Fairseq Model 11

4.1.2 CodeT5-small Model 13

4.1.3 Qwen2.5-Coder Model 15

5 Evaluation and Results **19**

5.0.1 BLEU Score Overview 19

5.0.2 Model Evaluation Results 20

5.0.3 Limitations of BLEU for Code Evaluation 20

5.0.4 LLM-Based Semantic Evaluation using Gemini 21

5.0.5 Evaluation Results 22

5.0.6 Why QwenCoder Outperforms Other Models 23

6 Conclusion and Future Work **25**

6.1 Conclusion 25

6.2 Future Work 26

References **29**

List of Figures

1.1 Illustration of a hardware Trojan: (A) Normal circuit, (B) Trojan module, (C) Trojan embedded in original circuit 3

3.1 Dataset Construction and Preprocessing Pipeline 8

4.1 Code Flow and evaluation 15

5.1 5 point similarity measure 22

List of Tables

5.1 BLEU Score Comparison Across Different Models 20

Chapter 1

Introduction

With the increasing integration of hardware components into critical systems, ensuring hardware security has become a major concern. This project focuses on identifying hardware Trojans by reverse engineering Verilog code into C for better analysis and detection.

1.1 Background

In the modern era of rapidly evolving digital infrastructure, hardware security has become a critical concern. With the increasing reliance on embedded systems, Internet of Things (IoT) devices, and third-party IP (Intellectual Property) cores, the integrity of hardware components is under constant threat. Among the most insidious threats are hardware Trojans—malicious modifications to circuit designs that can result in unauthorized control, sensitive data leakage, system malfunction, or even catastrophic failure.

Unlike software vulnerabilities, hardware Trojans are stealthy and often remain dormant until activated by specific internal or external conditions. These threats can be introduced at various stages of the hardware lifecycle, particularly during design and fabrication phases outsourced to untrusted third parties. Due to their low activation rate and stealthy behavior, traditional verification and testing techniques are often inadequate for

their detection.

At the same time, modern computer architecture is undergoing a fundamental shift toward co-processor-centric designs, with Field-Programmable Gate Arrays (FPGAs) playing a pivotal role. FPGAs offer key advantages such as:

Reconfigurability and parallel processing, enabling high performance and low-latency computation;

Customizability, useful for rapid prototyping and long-lifecycle industrial applications (e.g., aerospace, automotive, defense, and industrial automation);

Enhanced security, allowing the implementation of custom cryptographic primitives and secure boot mechanisms that are difficult to reverse-engineer or tamper with.

These benefits have led to the widespread adoption of FPGAs across security-sensitive domains. However, most commercial FPGA platforms—such as Xilinx SDNet, Netronome SDK, and Barefoot Capilano—rely on proprietary, closed-source compilers that translate high-level code (e.g., in C, C++, or Rust) into Verilog or VHDL. The opacity of these toolchains makes it difficult to verify the correctness and security of the resulting hardware descriptions.

This raises a critical challenge: how can we trust the FPGA-generated hardware designs without access to the internals of the compilation process? In particular, verifying the absence of hardware Trojans or logic obfuscation in synthesized FPGA bitstreams has become a central research problem in hardware security. Addressing this challenge requires robust reverse engineering and formal verification methods tailored to gate-level hardware.

To better understand the threat, Figure 1.1 illustrates a typical Trojan-infected hardware scenario. Section A shows a normal, unmodified circuit. Section B highlights the malicious Trojan module that is externally added. Section C demonstrates how this Trojan is stealthily inserted into the original circuit, making detection extremely difficult and posing a significant security threat.

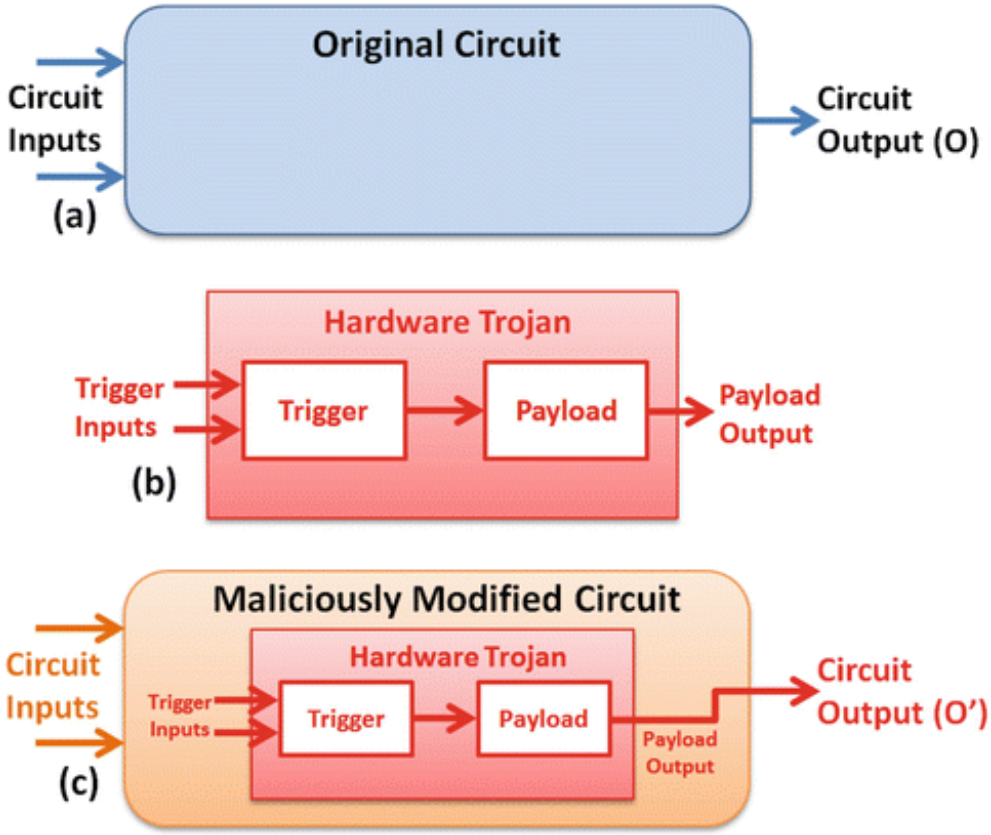


Fig. 1.1 Illustration of a hardware Trojan: (A) Normal circuit, (B) Trojan module, (C) Trojan embedded in original circuit

1.2 Contribution

To address this growing issue, reverse engineering techniques offer a promising approach. By analyzing low-level hardware descriptions, such as Verilog code, and translating them back into equivalent C representations, it becomes possible to reason about the intended functionality and uncover hidden or malicious behavior. This project explores the use of reverse engineering from Verilog to C as a means to facilitate the detection of hardware Trojans and enhance trust in digital hardware systems.

In addition to the core translation mechanism, the project contributes a structured methodology to identify anomalies during reverse engineering. It focuses on preserving functional integrity while mapping hardware constructs to software equivalents, enabling

easier analysis using familiar software-based tools. This work also highlights patterns commonly associated with Trojan behavior, such as redundant logic, unused signals, and suspicious control flows, thereby aiding future research and development of automated Trojan detection pipelines.

Chapter 2

Review of Prior Works

2.1 Review of Prior Work

1. Existing Approaches to Hardware Trojan Detection

Netlist extraction and analysis: Most existing approaches operate at the gate-level netlist stage, leveraging structural or functional analysis to detect anomalies. Several methods integrate machine learning techniques to enhance detection accuracy. For instance, the HAL framework automates netlist reverse engineering and Trojan identification through Boolean function analysis [H+21].

FPGA bitstream reverse engineering: Some techniques attempt to convert FPGA configuration bitstreams into gate-level netlists for static Trojan detection. These approaches highlight the significant challenges posed by proprietary bitstream formats and the computational complexity involved in scalable analysis.

RTL-level detection: Recent work by Sutikno et al. focuses on Register Transfer Level (RTL) Trojan detection using features extracted from Verilog conditional branching. By training decision tree classifiers on these features, they achieved a detection accuracy of 95.65% , showing promise in early-stage detection using source-level signals. [Seo18].

2. Verilog-to-C Conversion for Analysis

Verilator-based workflows: Some prior efforts employ Verilator to convert synthesizable Verilog into C++ and then into assembly code. These workflows use machine learning models, such as TextCNN, to analyze opcode streams and identify patterns indicative of Trojan activity. While this aligns partially with the goals of this project, those methods primarily focus on low-level mnemonics rather than semantically rich C-level abstractions.

Static vs. dynamic analysis: Whereas many studies emphasize static netlist-level analysis, the proposed approach explores the gap between RTL representations and software-level insight. By translating Verilog into C, this method leverages C's structural clarity to support more intuitive and accessible pattern recognition for security analysis.

For instance, the HAL framework [H+21] automates netlist reverse engineering...

These workflows use machine learning models, such as TextCNN [Seo18], to analyze opcode streams...

Chapter 3

Dataset Details

3.1 Dataset Construction and Preprocessing

Modern hardware design often requires the translation of high-level code into hardware description languages (HDLs) like Verilog. In our approach, we leveraged **Xilinx Vitis HLS** to convert C programs into synthesizable Verilog code. The objective of our dataset construction was to create a large corpus of C-Verilog code pairs to train and evaluate machine learning models for code translation and verification.

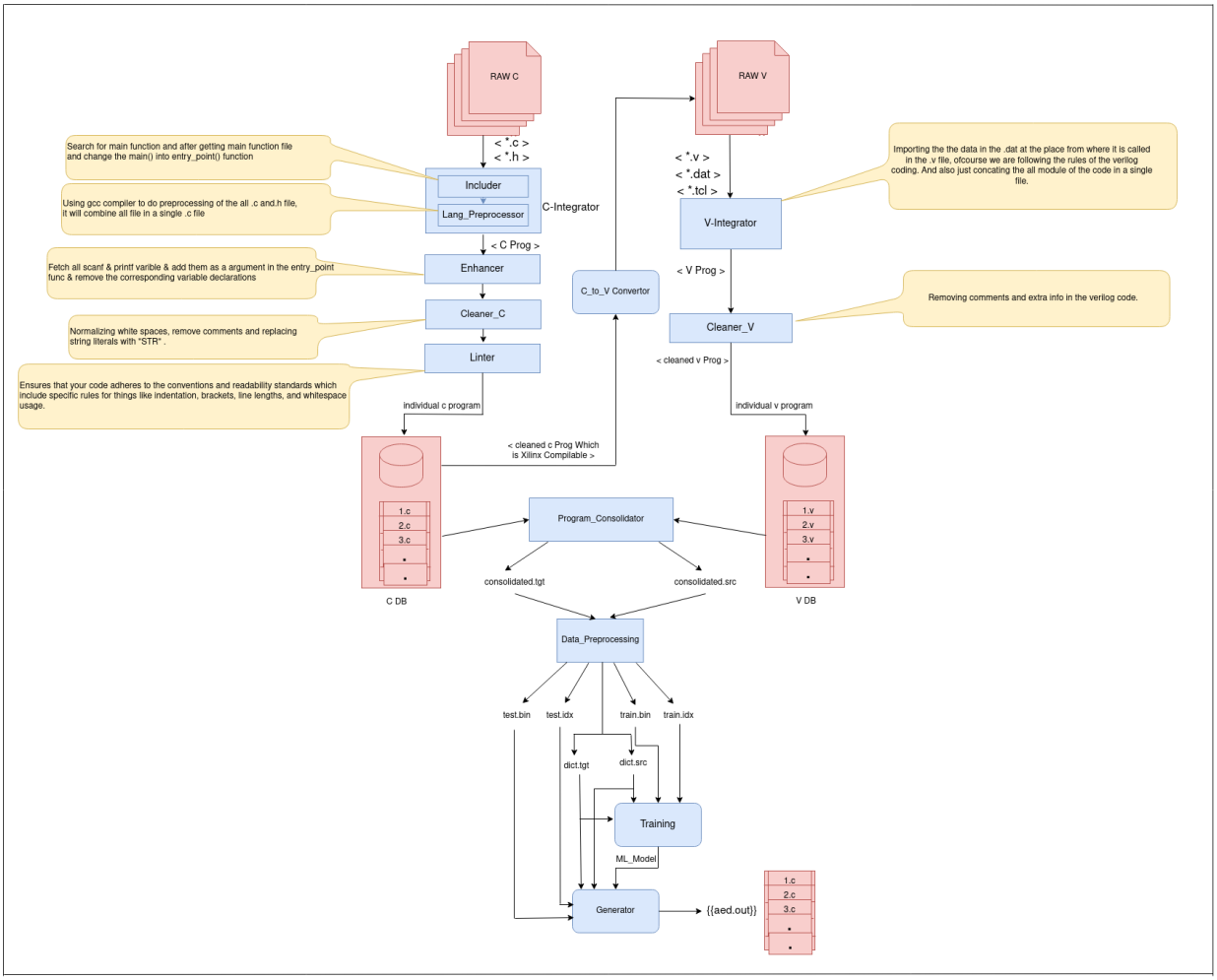


Fig. 3.1 Dataset Construction and Preprocessing Pipeline

As illustrated in Figure 3.1, the dataset construction process consists of the following stages:

3.1.1 C Code Preprocessing

- **Includer & Lang Preprocessor:** Aggregates all raw C source files and headers into a single file by performing preprocessing, including macro expansion and header inclusion.

- **Enhancer:** Identifies the `main()` function and transforms it into an `entry_point()` function. It also collects all `scanf` and `printf` variables and includes them as argu-

ments in the function signature.

- **Cleaner_C:** Performs normalization by removing comments, eliminating unnecessary white spaces, and replacing string literals with generic placeholders like `"STR"`.
- **Linter:** Ensures code style consistency by enforcing formatting rules related to indentation, brackets, line length, and whitespace usage.

3.1.2 Verilog Code Preprocessing

- **V-Integrator:** Imports data from auxiliary files such as `.dat`, `.tcl`, and includes all Verilog modules in a single file, following standard Verilog coding practices.
- **Cleaner_V:** Removes all comments and extraneous content to normalize the Verilog code for processing.

3.1.3 Program Consolidation and Data Preprocessing

Cleaned and preprocessed C and Verilog files are stored in separate databases and are consolidated into aligned code pairs by the **Program_Consolidator**. The **Data_Preprocessing** module tokenizes the code, generates training and test files (`*.bin`, `*.idx`), and constructs a dictionary source and target for supervised learning.

3.1.4 Model Training and Generation

The processed data is fed into a machine learning pipeline for training a translation model. The trained model then generates Verilog code from input C files, forming the foundation for further evaluation.

Chapter 4

ML Models

4.1 Main Architecture and Models Used

To accomplish the task of translating Verilog code to C code, we experimented with several advanced models designed for sequence generation and code understanding. One of the foundational components in our pipeline is the **Fairseq** framework, a sequence modeling toolkit developed by Facebook AI Research (FAIR). This section elaborates on how Fairseq was integrated and utilized in our system.

4.1.1 Fairseq Model

Fairseq is a high-performance framework built on PyTorch that supports a variety of sequence modeling tasks such as translation, summarization, and language modeling. It provides pretrained Transformer models as well as the flexibility to train custom models on user-defined data.

Data Representation and Tokenization

The Verilog-to-C translation task was framed as a sequence-to-sequence problem. The Verilog code was treated as the input sequence, and the corresponding C code was the

target sequence. Tokenization was performed using a pretrained tokenizer consistent with the architecture of the model being used.

For each input Verilog program, the tokenization and inference steps were executed as follows:

```
input_ids = tokenizer(input_text, return_tensors="pt",
                      max_length=512, truncation=True,
                      padding="max_length").input_ids.to(device)


output = model.generate(input_ids,
                        max_length=1024,
                        num_return_sequences=1,
                        do_sample=True,
                        temperature=0.7)
```

Explanation of Parameters:

- `max_length=512`: Limits the length of the input tokens to 512 for efficient processing.
- `padding="max_length"`: Ensures uniform sequence lengths across the batch.
- `do_sample=True`: Enables sampling during decoding instead of greedy search, improving output diversity.
- `temperature=0.7`: Controls randomness; lower values make the model more deterministic.
- `max_length=1024`: Specifies the maximum length for generated C code sequences.

Model Output

The output generated was decoded back into textual C code using the tokenizer's decode method. These generated outputs were then stored and compared with reference C codes

during evaluation.

4.1.2 CodeT5-small Model

CodeT5 is a Transformer-based model introduced by Salesforce Research specifically tailored for programming languages. It extends the T5 (Text-to-Text Transfer Transformer) architecture by training on a wide variety of programming tasks including code summarization, translation, and defect detection.

In our system, we utilized the **CodeT5-small** variant due to its efficiency and performance on low-resource machines, while still being capable of learning effective program mappings. The model was trained in a sequence-to-sequence fashion, where Verilog code was treated as the source sequence and corresponding C code as the target sequence.

Data Tokenization and Maximum Length

The input Verilog code and the corresponding target C code were tokenized using the pretrained tokenizer for CodeT5-small. To analyze the model's sensitivity to sequence length and its ability to retain contextual understanding, the model was executed twice with different token lengths:

- **Run 1**: Input token length of 512
- **Run 2**: Input token length of 1024

These experiments allowed us to evaluate the trade-off between model accuracy and computational efficiency. While the 512-token configuration ensures faster training and lower memory usage, the 1024-token configuration helps capture long-range dependencies in more complex Verilog programs.

Training Configuration

We trained the CodeT5-small model using the HuggingFace `Seq2SeqTrainer` API. The training hyperparameters were as follows:

```
training_args = Seq2SeqTrainingArguments(
    output_dir="./results",
    evaluation_strategy="epoch",
    learning_rate=5e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=3,
    weight_decay=0.01,
    logging_dir='./logs',
    save_total_limit=2,
    save_strategy="epoch",
    predict_with_generate=True,
    logging_strategy="steps",
    logging_steps=100,
    logging_first_step=True,
    report_to=["none"],
)
```

Explanation of Key Parameters

- `evaluation_strategy="epoch"`: Evaluates the model after every epoch for improved checkpointing.
- `learning_rate=5e-5`: A moderate learning rate suitable for small Transformer-based models.

- `batch_size=8`: Ensures training can fit on standard GPUs.
- `num_train_epochs=3`: Balances performance with training time.
- `predict_with_generate=True`: Enables sequence generation during evaluation.
- `logging_steps=100`: Provides regular feedback during training.

Model Output and Integration

During inference, the trained CodeT5-small model was used to translate cleaned Verilog inputs into their respective C code representations. The generated sequences were later evaluated against ground-truth C programs for semantic and syntactic correctness. This model served as a strong baseline for code generation, especially due to its lightweight footprint and fast convergence.

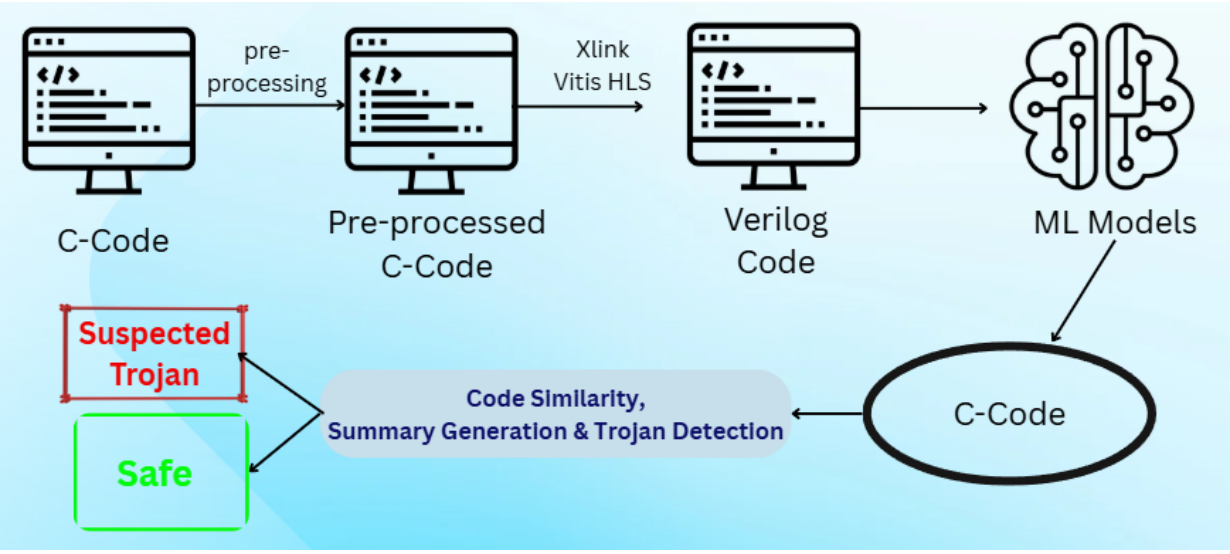


Fig. 4.1 Code Flow and evaluation

4.1.3 Qwen2.5-Coder Model

The `Qwen2.5-Coder-1.5B-Instruct` model, developed by Alibaba, was used for Verilog-to-C code translation using a prompt-based few-shot learning strategy [Bro20]. This model

is specifically designed for code understanding and generation tasks, making it a suitable candidate for our translation use case.

The generation parameters used with this model were:

- **Maximum new tokens**: 1024
- **Temperature**: 0.7
- **Sampling enabled**: True
- **Type:** Causal Language Models
- **Training Stage:** Pretraining & Post-training
- **Architecture:** Transformers with RoPE, SwiGLU, RMSNorm, Attention QKV bias, and tied word embeddings
- **Number of Parameters:** 1.54B (1.31B non-embedding)
- **Number of Layers:** 28
- **Number of Attention Heads (GQA):** 12 for Q and 2 for KV
- **Context Length:** Full 32,768 tokens

The translation process was guided using a few-shot prompt that included an example Verilog-to-C mapping, followed by a new Verilog snippet to be translated. The prompt was carefully structured to ensure the model's output adhered to a consistent format, avoided extraneous explanations, and maintained variable consistency.

Listing 4.1 Qwen2.5-Coder Model Configuration and Prompt Setup

```
model_name = "Qwen/Qwen2.5-Coder-1.5B-Instruct"
generate_kwargs = {
    'max_new_tokens': 1024,
```

```
    'temperature': 0.7,
    'do_sample': True
}

prompt = (
    "Translate the following Verilog code to C code. "
    "Use the provided example mappings as a reference for style and struct
    "Only output the C code, without any explanations, comments, or additi
    "Ensure the variables in the generated C code match those used in the
    "Avoid including unused variables or constructs.\n"
    "Example Input and Output:\n"
    "Verilog Code:\n"
    f"{sample_verilog}\n\n"
    "Corresponding C Code:\n"
    f"{sample_c_code}\n\n"
    "Verilog Code to Translate:\n"
    f"{verilog_code}"
)
```

This method leverages the model's instruction-following capability and provides contextual guidance through in-context examples, resulting in significantly improved translation consistency.

Chapter 5

Evaluation and Results

5.0.1 BLEU Score Overview

BLEU (Bilingual Evaluation Understudy) is a widely-used metric for evaluating the quality of text which has been machine-translated from one language to another. It compares n-grams of the candidate translation to one or more reference translations and calculates a precision score, adjusted by a brevity penalty to penalize overly short translations.

Formula

The BLEU score is computed as:

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right)$$

Where:

- p_n = precision for n-grams (i.e., fraction of n-grams in the candidate that appear in the reference),
- w_n = weights for each n-gram order (typically uniform),

- BP = brevity penalty, computed as:

$$\text{BP} = \begin{cases} 1, & \text{if } c > r \\ e^{(1-r/c)}, & \text{if } c \leq r \end{cases}$$

where c is the length of the candidate translation and r is the effective reference length.

5.0.2 Model Evaluation Results

Model Name	Max Length Hyperparameter	BLEU Score
Fairseq	512	0.452
CodeT5 (512)	512	0.458
CodeT5 (1024)	1024	0.467
Qwen Coder	—	0.560

Table 5.1 BLEU Score Comparison Across Different Models

5.0.3 Limitations of BLEU for Code Evaluation

While BLEU is suitable for evaluating natural language translation tasks, it has notable limitations in the context of programming language tasks. Specifically:

- BLEU focuses on surface-level token similarity rather than semantic or syntactic correctness.
- Code has structural and logical dependencies that BLEU cannot capture (e.g., variable renaming or equivalent logical expressions).
- It is possible for two code snippets with the same functionality to yield a low BLEU score due to different token arrangements.

Hence, BLEU should be used with caution in code generation tasks, and ideally supplemented or replaced with more code-specific metrics such as CodeBLEU, Exact Match, or execution-based evaluation.

5.0.4 LLM-Based Semantic Evaluation using Gemini

To overcome the limitations of BLEU in assessing the functional and semantic equivalence of code, we employed a large language model—**Gemini**—to evaluate the similarity between the predicted C code (generated from Verilog) and the actual ground truth C code.

Our approach involved the following steps:

1. First, we generated summaries of both the translated C code and the actual C code using Gemini.
2. Then, we crafted a prompt that includes both summaries and their corresponding full code snippets.
3. Gemini was asked to:
    - Provide a **similarity score from 1 to 5**, where 5 indicates very high similarity.
    - Give a short descriptive justification of the score.
    - Consider functionality and output while rating similarity, even if the structure differs.
    - Indicate any potential threat or anomaly (e.g., **Trojan**) based on deviations in semantics.

Prompt Used

```
How similar are the two codes? I have provided summary and code both.
One of the summary is of the code which is translated from the Verilog code to C
and other is the actual C code for that.

First, give a similarity score between 1 to 5 as 5 point measure of similarity,
then a brief description. Even if the functionality or output are similar consider th
```

```
Code1 Summary: {code1}

Code2 Summary: {code2}

Code1: {main_code1}

Code2: {main_code2}
```

This method allowed us to qualitatively and semantically compare the codes beyond surface-level syntax. Moreover, by asking for anomaly identification, we were able to flag any suspicious divergence in logic or structure that might indicate the presence of a **Trojan or malicious behavior**.

5.0.5 Evaluation Results

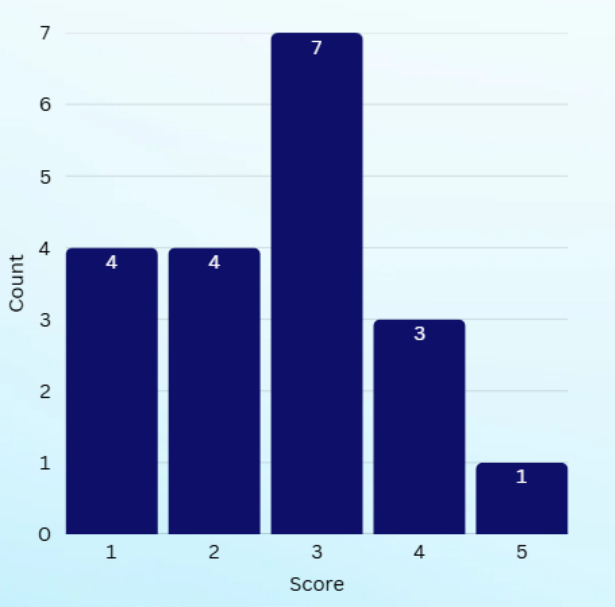


Fig. 5.1 5 point similarity measure

We evaluated the translated C code using a 5-point similarity scoring method [Bro10] powered by the Gemini model. The results were encouraging—more than **50%** of the

translated samples received a similarity score of **3 or higher**, indicating strong semantic alignment with the reference (ground truth) C code.

Furthermore, we tested the system on the **HT (Hardware Trojan) database**, which contains Verilog designs with injected Trojans. Of 10 samples injected with Trojans [zf24], the system successfully flagged **7 samples** as potentially malicious. This demonstrates the system's practical effectiveness in identifying anomalous behavior and its viability for **real-world Trojan detection** scenarios.

5.0.6 Why QwenCoder Outperforms Other Models

The superior performance of **QwenCoder** over other baseline models can be attributed to the following key factors:

- **Higher Maximum Token Length:** QwenCoder supports significantly longer input sequences, allowing it to process large Verilog modules without truncation. This capability is crucial for preserving structural and contextual information, which is often lost in models with shorter token limits.

- **Large-scale Training:** QwenCoder is pretrained on a massive and diverse dataset containing source code, synthetic data, and text-code grounded examples. This enables the model to generalize more effectively across both low-level hardware description languages (like Verilog) and high-level programming languages (such as C).

- **Enhanced Cross-language Understanding:** The model is specifically optimized for multi-language code tasks, giving it a strong ability to bridge the abstraction gap between hardware (Verilog) and software (C). This cross-domain alignment makes it particularly effective in translation and reasoning tasks across languages.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

In this work, we proposed a novel methodology for detecting Hardware Trojans (HTs) in digital circuits by leveraging the power of large language models (LLMs) and reverse engineering techniques. Our approach focused on translating Verilog hardware description code into semantically equivalent C code using advanced code language models, notably **QwenCoder**, which demonstrated superior performance in terms of cross-language understanding and semantic preservation.

The motivation behind converting Verilog to C lies in the richer ecosystem of software-level analysis tools and the availability of semantic comparison techniques in the software domain. By enabling this translation, we could apply high-level reasoning to low-level hardware constructs, thus opening new possibilities for Trojan detection.

To validate the translated code, we employed a combination of BLEU scoring and LLM-based semantic evaluation. While BLEU gave us a rough syntactic similarity, we found that it often failed to capture the functional correctness of the code. Therefore, we integrated the **Gemini LLM** to perform semantic comparisons between the predicted C code and the ground truth. Gemini was asked to provide a 5-point similarity score along with an explanation and detect any functional anomalies indicative of malicious behavior.

Our evaluation showed that over 50% of the translated samples achieved a similarity score of 3 or more, confirming the functional fidelity of the translated C code. More significantly, when tested on the HT database containing Trojan-injected Verilog samples, our system successfully flagged 7 out of 10 cases as suspicious. This demonstrates the potential of our pipeline as a practical solution for detecting stealthy hardware Trojans, which often evade traditional signature-based detection mechanisms.

The use of QwenCoder was instrumental in this success, primarily due to its:

- Support for longer input sequences, allowing untruncated analysis of complex Verilog modules.
- Pretraining on a large and diverse corpus of code, which helps in generalizing across hardware and software languages.
- Strong cross-language capabilities, enabling accurate reverse engineering between Verilog and C.

Overall, this project represents a step forward in hardware security research, combining software-level reasoning, reverse engineering, and the latest advancements in AI-driven code understanding. Future work could involve integrating this system into automated hardware verification workflows or extending it to detect other classes of hardware vulnerabilities.

6.2 Future Work

To further enhance the effectiveness and reliability of the proposed system, several improvements are envisioned:

- **Advanced Similarity Evaluation:** Move beyond BLEU and summaries by incorporating AST-based comparison, embedding-based similarity (e.g., CodeBERT), and dynamic execution testing to better capture functional equivalence between the predicted and actual C code.

- **Improved Trojan Detection:** Instead of relying solely on summaries, future methods should analyze actual code behavior through symbolic execution, dataflow tracking, and execution trace comparison for precise detection of stealthy Trojans.
- **Support for Other HDLs:** Extend the system to handle other hardware description languages like VHDL or SystemVerilog to increase applicability across a wider range of hardware designs.
- **Integration with Toolchains:** Embed the solution into existing EDA workflows to enable real-time security analysis and early detection of malicious logic during hardware design.

References

[Bro10] Sorrel Brown. Likert scale examples for surveys. https://www.extension.iastate.edu/documents/anr/likertscaleexamplesforsurveys.pdf, 2010.

[Bro20] Tom B. Brown. Language models are few-shot learners. https://arxiv.org/pdf/2005.14165, 2020.

[H+21] Michael H. et al. Hal – the missing piece of the puzzle: Formal hardware reverse engineering using gate-level abstraction. https://eprint.iacr.org/2021/1278.pdf, 2021. IACR Cryptology ePrint Archive.

[Seo18] Yezee Seo. Hardware trojan detection via opcode sequences from verilog. https://www.ndss-symposium.org/wp-content/uploads/2018/02/ndss2018posters$_p aper_1$6.*pdf*, 2018.$NDSSSymposiumPoster$.

[zf24] zhenyu fan. Hardware trojan database. https://www.scidb.cn/en/detail?dataSetId=151ed5eb8 2024.